

Class 10 Class Program :- ADC With PWM

```
/*
 * File:  main.c
 * Author:  sagar
 *
 * Created on 2 June, 2025, 10:49 AM
 */

#include <xc.h>
#include<stdint.h>
#include<stdio.h>
#define _XTAL_FREQ 6000000

/* Function prototypes */
void pwmInit(void);
void adcLcdInit(void);
void LcdOutput(uint16_t i);
void LcdCommand(uint8_t i);
void LcdData(uint8_t i);

uint16_t map(long x, long in_min, long in_max, long out_min, long out_max);

/* Used variables */
uint16_t adcHigh, adcLow, calibratedValue, adcVolt;
char calibratedVolt[11] = "ADC VALUE: ";

/* main function */
void main(void)
{
    pwmInit();
    adcLcdInit();
    LcdCommand(0x80);
    for (int i = 0; i < 10; i++)
    {
        LcdData(calibratedVolt[i]);
    }
    while (1)
    {
        // Start ADC conversion
        ADCON0 |= 0x04;
        while (ADCON0 & 0x04) {} // Wait for conversion to complete

        // Read ADC result (10-bit value)
        adcHigh = ADRESH;
        adcLow = ADRESL;
        adcVolt = ((adcHigh << 8) | adcLow); // Right justified result

        // Map ADC value to 0-338 range
        calibratedValue = map(adcVolt, 0, 1023, 0, 338);
    }
}
```

```

    // Correct PWM Register configuration (10-bit PWM)
    CCPR1L = (uint8_t)(calibratedValue >> 2); // Upper 8 bits
    CCP1CON = (CCP1CON & 0xCF) | ((calibratedValue & 0x03) << 4); // Lower 2 bits in
    CCP1CON<5:4>

    // Display mapped value on LCD
    LcdCommand(0x8A); // Move LCD cursor to proper position
    LcdOutput(calibratedValue);

    __delay_ms(200); // Optional delay for LCD stability
}
return;
}

```

```

void pwmInit(void)
{
    TRISC = 0xFB; // Make RC2 (CCP1) an output

    CCP1CON = 0x0C; // PWM mode, 10-bit
    T2CON = 0x06; // Timer2 ON, prescaler 1:16
    PR2 = 0x5E; // Set period

    TMR2 = 0; // Reset Timer2 counter
    TMR2ON = 1; // Turn on Timer2
}

```

```

/* ADC and LCD init function */
void adcLcdInit(void)
{
    TRISB = 0x00; // Set PORTC as output (for control signals)
    TRISD = 0x00;
    PORTB = 0x00;
    PORTD = 0x00;

    /******* LCD init *****/
    // LCD initialization sequence as per the HD44780 LCD datasheet
    LcdCommand(0x30); // Send function set command (8-bit mode)
    __delay_ms(100); // Delay for command execution
    LcdCommand(0x30); // Repeat function set command
    __delay_ms(100);
    LcdCommand(0x30); // Repeat function set command again
    __delay_ms(100);
    LcdCommand(0x38); // Set LCD for 8-bit mode, 2-line display, 5x8 font
    __delay_ms(100);
    LcdCommand(0x0C); // Turn on display, cursor off
    __delay_ms(100);
    LcdCommand(0x01); // Clear the display
    __delay_ms(100);
}

```

```

/***** ADC Init *****/
ADCON1 = 0x8E;
ADCON0 = 0x81;
__delay_ms(10);
}

/* map function to calibrate the values */
uint16_t map(long x, long in_min, long in_max, long out_min, long out_max)
{
    return (uint16_t) (((x - in_min) * (out_max - out_min)) / (in_max - in_min) + out_min);
}

/* LCD output function */
void LcdOutput(uint16_t i)
{
    uint8_t d1, d2, d3, d4; // Creating local var to reduce memory consumption
    d3 = (uint8_t) ((i % 1000) / 100); // Extract hundreds place
    d2 = (uint8_t) ((i % 100) / 10); // Extract tens place
    d1 = (uint8_t) (i % 10); // Extract ones place

    LcdCommand(0x8A);
    LcdData(0x30 + d3);
    LcdData(0x30 + d2);
    LcdData(0x30 + d1);
}

/* Function to send data (characters) to the LCD */
void LcdData(uint8_t i)
{
    PORTB |= (0x1 << 3); // Set RS (RC3) = 1 (indicates data mode)
    PORTD = i; // Place data on PORTD
    PORTB |= (0x1 << 0); // Set EN (RC0) = 1 (enable pulse start)
    PORTB &= ~(0x1 << 0); // Set EN (RC0) = 0 (enable pulse end)
    __delay_ms(100); // Small delay for command execution
}

/* Function to send commands to the LCD */
void LcdCommand(uint8_t i)
{
    PORTB &= ~(0x1 << 3); // Set RS (RC3) = 0 (indicates command mode)
    PORTD = i; // Place command on PORTD
    PORTB |= (0x1 << 0); // Set EN (RC0) = 1 (enable pulse start)
    PORTB &= ~(0x1 << 0); // Set EN (RC0) = 0 (enable pulse end)
    __delay_ms(100); // Small delay for command execution
}

```